



PRESENTS

DATAFORGE

TRACK 1: The "Hallucination Hunter": Automated Fact-Checking & Attribution for LLMs

Real-World Context

Large Language Models (LLMs) like GPT-4, Claude, and LLaMA have revolutionized content generation, from drafting legal contracts to summarizing medical records. However, these models suffer from a critical flaw: **Hallucination**. They can confidently generate plausible-sounding but factually incorrect information.

In high-stakes domains—such as **Healthcare** (summarizing patient history), **Law** (citing non-existent precedents), or **Finance** (fabricating earnings report numbers)—a single hallucination can lead to malpractice lawsuits, financial loss, or dangerous operational failures. Organizations currently rely on expensive manual review to verify AI-generated content, which defeats the purpose of automation.

Why This Is a Real Problem

Trust is the bottleneck for Enterprise AI adoption. Users cannot fully rely on an AI assistant if they have to double-check every sentence. Major challenges include:

- **Stochasticity:** LLMs generate different answers every time; facts can "drift."
- **Source Amnesia:** Models often cannot point to *where* they learned a fact.
- **Subtle Errors:** A summary might be 99% correct but flip a single crucial detail (e.g., changing "Type A" to "Type B"), which is harder to detect than obvious gibberish.
- **Verification Latency:** Manually cross-referencing a 10-page summary against 50 pages of source documents is humanly exhausting.

Enterprises need an "AI Auditor"—a secondary system that acts as a guardrail, verifying generated text against a trusted Knowledge Base (KB) before it reaches the human user.

What You Are Expected to Build

Design an AI-assisted **Fact-Checking & Citation System** that:

1. **Ingests** a set of "Trusted Source Documents" (Ground Truth) and an "LLM-Generated Summary/Answer."
2. **Decomposes** the generated text into atomic factual claims.
3. **Verifies** each claim against the Source Documents using RAG (Retrieval-Augmented Generation) or NLI (Natural Language Inference) techniques.
4. **Classifies** claims into categories: *Supported, Contradicted, or Unverifiable*.
5. **Annotates** the text with citations (linking specific claims to specific source paragraphs).
6. **Calculates** an overall "Trust Score" for the document.

Expected Inputs (You May Assume)

- **Source Knowledge Base:** A set of PDF/Text documents (e.g., medical guidelines, legal acts, or technical manuals).
- **LLM Output:** A generated summary or answer based on those documents.
- **Ground Truth Annotations (for training/testing):** Examples of sentences marked as "Hallucinated" or "Faithful" with reasoning.
- *Note: Synthetic datasets (like HaluEval or FEVER) can be used as proxies.*

Expected Outputs

1. **Annotated Document**
 - a. The original LLM output with color-coded highlighting:
 - i. **Green:** Factually supported by sources.
 - ii. **Red:** Hallucination (Directly contradicts source).
 - iii. **Yellow:** Not enough information / Unverifiable.
2. **Citation Report**
 - a. For every "Green" sentence, provide a direct link/snippet to the exact paragraph in the Source Document that validates it.
3. **Correction Engine**

- a. For every "Red" sentence, suggest a corrected version based on the source text.

4. Visualization of Verification (Mandatory)

- a. A split-screen or interactive interface.
- b. Clicking a sentence in the *Generated Text* should auto-scroll the *Source Document* view to the relevant evidence.
- c. Visual "Confidence Meter" for the document.

Explainability Requirement (Core of This Problem)

Your system must answer:

- **Why is this flagged as a hallucination?** (e.g., "The summary states the patient has Type 2 Diabetes, but Source Document B, Page 4 states the patient is non-diabetic.")
- **Where is the proof?** (Exact source passage extraction).
- **How confident is the model?** (Distinguish between a 'likely' error and a 'certain' error).

Explanations must be concise and actionable for a human reviewer.

What Is NOT Expected

- **General Internet Search:** You are verifying against the *provided documents*, not the entire web (unless specified as an extension).
- **Perfect Grammar Checking:** Focus on *factual consistency*, not style.
- **Retraining Foundation Models:** You should use lightweight models (BERT, RoBERTa) or API calls for the verification step; do not try to pre-train a GPT-4 competitor.

How This Will Be Evaluated

- **Detection Accuracy:** F1-Score on identifying hallucinations (especially avoiding False Negatives—missing a lie is worse than flagging a truth).
- **Citation Precision:** Does the citation actually support the claim?
- **Explanation Quality:** Are the generated reasons coherent?
- **UI/UX:** How easy is it for a human to review the document using your tool?
- **Latency:** Can this run in near real-time (under 5 seconds for a page)?

TRACK 2: AI-Powered Intelligent Data Migration Intelligence

Real-World Context

Organizations frequently migrate data between databases due to cloud modernization, legacy system upgrades, ERP/CRM replacements, mergers and acquisitions, and technology consolidation.

During such migrations, data must move from a Source Database to a Target Database, often with:

- Different schemas
- Renamed columns
- Changed data types
- Split or merged fields
- Different constraints and relationships

Traditional migration approaches rely on manual schema comparison, hard-coded scripts, and static rules, which are error-prone and difficult to audit.

Why This Is a Real Problem

Incorrect data migration can cause financial inconsistencies, reporting errors, broken relationships, regulatory non-compliance, and production outages.

Major challenges include:

- Identifying semantic column matches when names differ
- Handling data format transformations
- Detecting partial or incorrect loads
- Explaining mapping decisions
- Providing visual clarity of schema relationships

Enterprises need intelligent systems that understand schema meaning, suggest mappings, detect inconsistencies, and provide visual audit trails.

What You Are Expected to Build

Design an AI-assisted data migration system that:

1. Connects to Source and Target databases
2. Discovers schema structure
3. Suggests intelligent column mappings
4. Executes migration safely
5. Validates migrated data
6. Generates explainable reports
7. Provides clear visualization of table and column mappings

Your system should not only move data — it should understand, explain, and visualize the migration process.

Expected Inputs (You May Assume)

- Source database schema and sample data
- Target database schema
- Tables with renamed columns, different data types, format differences, and possible one-to-many or many-to-one relationships

Synthetic datasets may be generated if required.

Expected Outputs

1. Migrated Dataset
 - Data successfully transferred to the target database.
2. Mapping Report
 - Source-to-target column mappings
 - Confidence or similarity scores
 - Transformation logic applied
3. Validation Report
 - Row count comparison
 - Null and duplicate checks



- Referential integrity validation
 - Failed record reporting
4. Visualization of Table Mapping (Mandatory)

- Clear representation of source and target tables
- Mapping relationships (1:1, 1:Many, Many:1)
- Highlight unmapped fields
- Optional confidence indicators

Visualization may be an interactive dashboard, ER-style diagram, Sankey diagram, or graph-based flow.

Explainability Requirement (Core of This Problem)

Your system must answer:

- Why was this column mapped to that column?
- Why was another column ignored?
- What transformation was applied?
- What data failed and why?

Explanations must be understandable by non-technical stakeholders.

What Is NOT Expected

- Production-grade migration tooling
- Perfect mapping accuracy
- Large enterprise datasets

Focus on **intelligent mapping, explainability, validation, and visualization.**

How This Will Be Evaluated

- Clarity of mapping logic
- Quality of explanations
- Accuracy of migration
- Validation robustness
- Visualization clarity
- Practical enterprise relevance

TRACK 3: Retrieval Integrity Auditor for RAG Systems

Real-World Context

Retrieval-Augmented Generation (RAG) systems retrieve documents before generating answers. However, retrieval failures often occur in subtle ways:

- Retrieve irrelevant documents
- Miss critical documents
- Over-rely on top-1 similarity
- Hide retrieval errors behind fluent generation

Today, most systems evaluate only the final answer, but rarely evaluate the retrieval step itself. Enterprises need visibility into whether the correct evidence was retrieved before generation.

Why This Is a Real Problem

Retrieval failures can lead to incomplete or misleading answers even when the language model appears fluent. Key risks include compliance errors, operational failures, and decision-making based on partial evidence.

Major challenges include:

- No measurement of evidence coverage
- Difficulty detecting missing critical context
- Inability to identify retrieval noise
- Lack of explainability in retrieval ranking decisions

What You Are Expected to Build

Design a Retrieval Integrity Auditor that:

1. Audits retrieval outputs from a RAG system
2. Measures coverage of relevant knowledge
3. Detects missing critical evidence
4. Scores retrieval completeness
5. Explains retrieval failures and improvement suggestions



Your system must generate both a machine-readable audit output (e.g., JSON) and a human-readable retrieval integrity report.

Expected Inputs (You May Assume)

- Knowledge Base: Collection of documents (PDF/Text) with optional metadata
- Retrieval Output: Top-k retrieved chunks with rank and similarity scores
- User Query
- Optional Ground Truth: Labeled relevant documents/chunks for evaluation

Expected Outputs

1. Retrieval Integrity Score (0–100)
2. Coverage Analysis (which query aspects are supported or missing)
3. Evidence Classification (supporting vs noise chunks)
4. Missing Evidence Detection (likely relevant documents not retrieved)
5. Improvement Recommendations (e.g., query rewrite, hybrid retrieval, re-chunking)

Mandatory Visualization:

- Aspect vs Chunk coverage matrix or heatmap
- Highlight missing aspects
- Clickable drill-down to evidence

Explainability Requirement (Core of This Problem)

Your system must answer:

- Why are the retrieved documents insufficient?
- Which aspect of the query is missing and why does it matter?
- Which documents should likely have been retrieved?
- What concrete changes would improve retrieval?

Explanations must be clear, structured, and understandable by non-technical reviewers.

What Is NOT Expected

- Training new embedding models from scratch
- Perfect retrieval accuracy

- Full production RAG platform

Focus on auditing retrieval quality, measuring coverage, and explaining failures.

How This Will Be Evaluated

- Coverage Detection Accuracy
- Missing Evidence Identification
- Noise Detection Precision
- Quality of Explanations
- Visualization Clarity
- Practical Feasibility for enterprise workflows





TRACK 4: Contextual Compression for Extreme Long Inputs

Real-World Context

Enterprises deal with very large content every day, like:

- 500–5,000 page policy documents
- Long audit logs and incident reports
- Customer support transcripts
- Technical manuals and runbooks
- Legal / compliance archives

But most AI systems (and even humans) cannot read everything at once. LLMs have a **limited context window**, so they can't "fit" huge documents directly.

Why This Is a Real Problem

The common solution is "just summarize", but that fails because:

- Important details get lost
- Summaries hide exceptions, risks, and contradictions
- You can't trace answers back to the original source
- Different teams summarize differently → inconsistency
- "Compressed" content becomes untrustworthy for decision-making

Enterprises need compression that preserves:

decision-critical facts

traceability (where did this come from?)

contradictions and edge cases

What You Are Expected to Build

Design a **Contextual Compression Engine** that:

- Takes a very large document set (or long single document)
- Compresses it step-by-step (not one-shot summarization)
- Produces a compressed representation that still supports:
 - Question answering
 - Drill-down to original text
 - Evidence / citations

- Provides a way to estimate or explain information loss

Think of it like:

“A smart compression system that keeps what matters and proves it.”a

Expected Inputs (You May Assume)

- A large document (PDF/text) OR multiple text files
- Optional: document metadata (date, author, category)

Teams may use:

- Synthetic documents (generated)
- Public documents
- Logs / transcripts (dummy)

Expected Outputs

Your system should produce:

Compressed Output

A compact representation such as:

Multi-level summaries (section → chapter → full)

“Key facts + exceptions + risks” list

Structured notes (JSON)

Traceability Links

Every important point should be traceable:

“This claim came from Section 3.2, Paragraph 4”

Support for Drill-Down

A user should be able to:

Start with compressed summary

Expand specific parts when needed

Example:

“Show me details behind this risk item.”

Key Ideas

Hierarchical Compression

Do not summarize the entire document at once.

Instead:

- Break into chunks/sections
- Summarize each section
- Summarize summaries
- Maintain structure

Preservation of “Decision-Critical” Content

Teach your system to keep:

- Numbers, thresholds, limits
- Dates
- Exceptions (“unless...”, “only if...”)
- Risks and constraints
- Contradictions (“policy A says X, policy B says Y”)

Traceability (Non-Negotiable)

- Each compressed statement should include:
- Source reference (section, paragraph, line range, doc ID)

Explainability Requirement (Core of This Problem)

Your output should answer:

- Why was this included in the compression?
- What was removed and why?
- Can I trace important points back to the original?

A compressed answer without traceability is not acceptable for enterprise use.

What Is NOT Expected

- Perfect compression quality
- Handling millions of pages
- Training new LLMs
- Fancy UI

Focus on **structure, traceability, and decision-preserving compression**.

How This Will Be Evaluated

Judges will look for:

- Clear compression strategy (hierarchical design)
- Evidence/traceability quality
- Handling of edge cases (exceptions/contradictions)
- Practical feasibility for enterprise adoption

